

Chapter 1: Introduction

- Purpose of Database Systems
- View of Data
- Data Models
- Data Definition Language
- Data Manipulation Language
- Transaction Management
- Storage Management
- Database Administrator
- Database Users
- Overall System Structure

Database Management System (DBMS)

- Collection of interrelated data
- Set of programs to access the data
- DBMS contains information about a particular enterprise
- DBMS provides an environment that is both *convenient* and *efficient* to use.
- Database Applications:
 - Banking: all transactions
 - Airlines: reservations, schedules
 - Universities: registration, grades
 - Sales: customers, products, purchases
 - Manufacturing: production, inventory, orders, supply chain
 - Human resources: employee records, salaries, tax deductions
- Databases touch all aspects of our lives

Purpose of Database System

- In the early days, database applications were built on top of file systems
- Drawbacks of using file systems to store data:
 - Data redundancy and inconsistency
 - ✦ Multiple file formats, duplication of information in different files
 - Difficulty in accessing data
 - ✦ Need to write a new program to carry out each new task
 - Data isolation — multiple files and formats
 - Integrity problems
 - ✦ Integrity constraints (e.g. account balance > 0) become part of program code
 - ✦ Hard to add new constraints or change existing ones

Purpose of Database Systems (Cont.)

- Drawbacks of using file systems (cont.)

- Atomicity of updates

- ✦ Failures may leave database in an inconsistent state with partial updates carried out
 - ✦ E.g. transfer of funds from one account to another should either complete or not happen at all

- Concurrent access by multiple users

- ✦ Concurrent accessed needed for performance
 - ✦ Uncontrolled concurrent accesses can lead to inconsistencies
 - E.g. two people reading a balance and updating it at the same time

- Security problems

- Database systems offer solutions to all the above problems

Levels of Abstraction

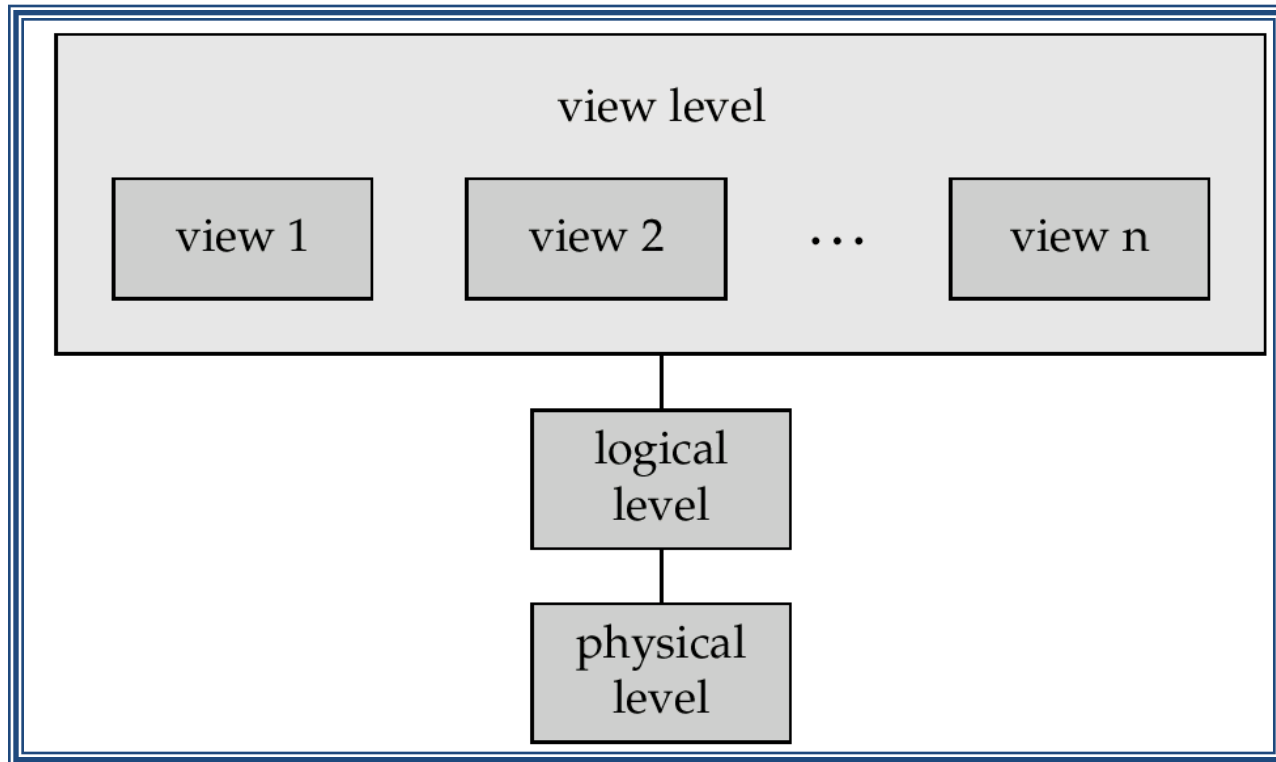
- Physical level describes how a record (e.g., customer) is stored.
- Logical level: describes data stored in database, and the relationships among the data.

```
type customer = record  
    name : string;  
    street : string;  
    city : integer;  
end;
```

- View level: application programs hide details of data types. Views can also hide information (e.g., salary) for security purposes.

View of Data

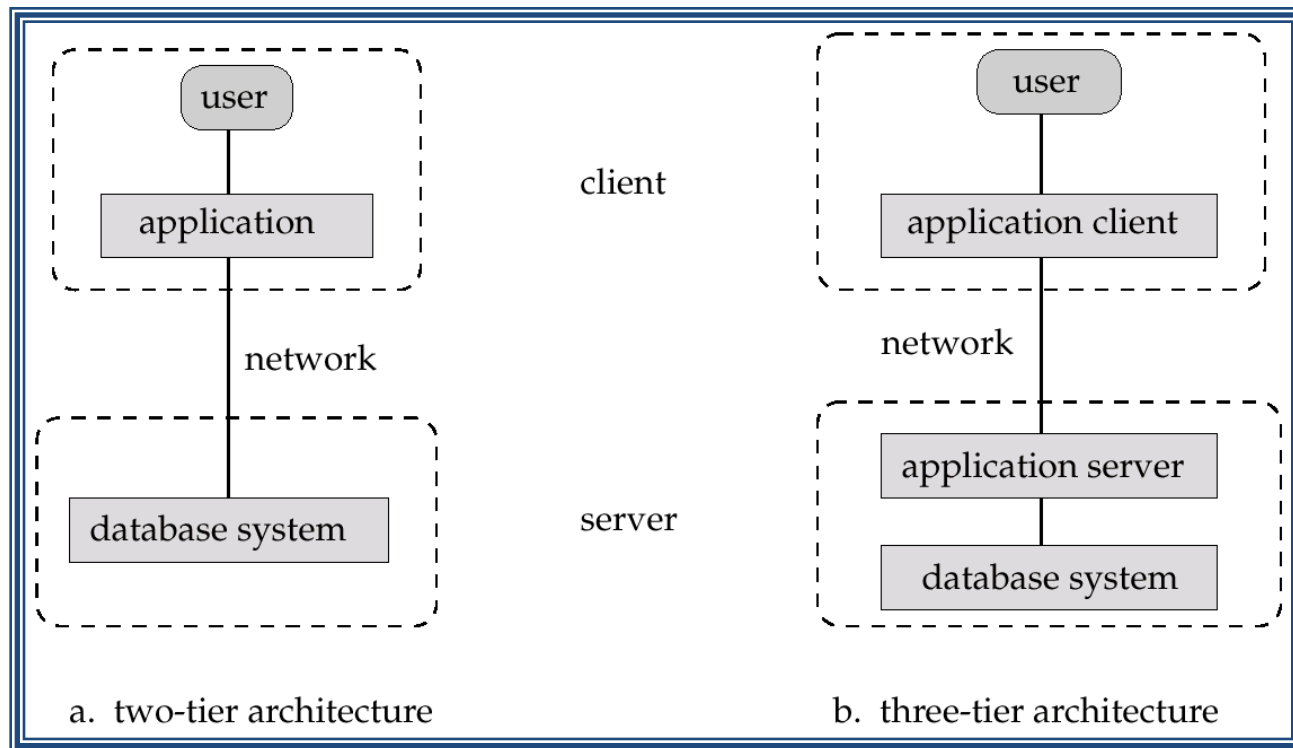
An architecture for a database system



Instances and Schemas

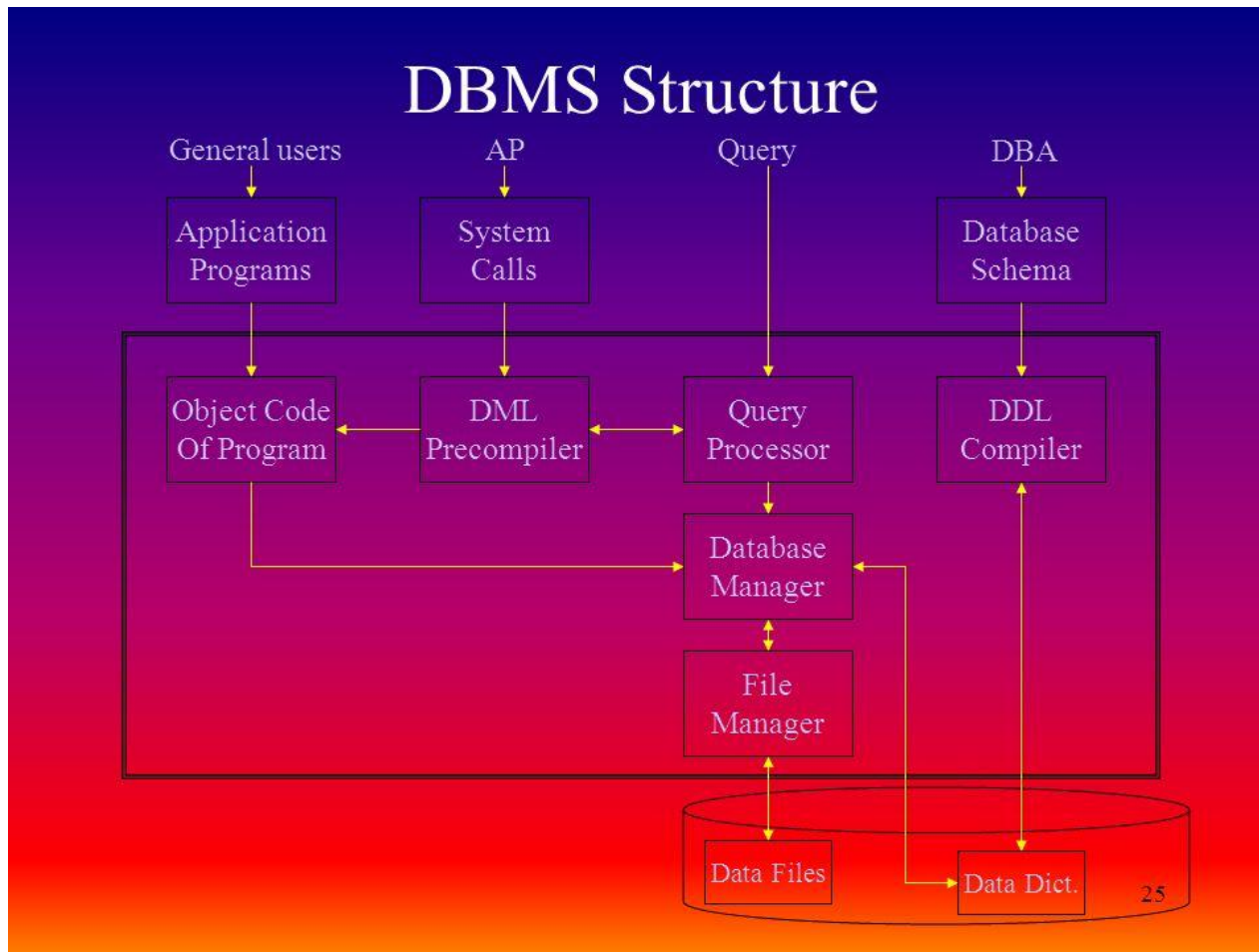
- Similar to types and variables in programming languages
- **Schema** – the logical structure of the database
 - e.g., the database consists of information about a set of customers and accounts and the relationship between them)
 - Analogous to type information of a variable in a program
 - **Physical schema**: database design at the physical level
 - **Logical schema**: database design at the logical level
- **Instance** – the actual content of the database at a particular point in time
 - Analogous to the value of a variable
- **Physical Data Independence** – the ability to modify the physical schema without changing the logical schema
 - Applications depend on the logical schema
 - In general, the interfaces between the various levels and components should be well defined so that changes in some parts do not seriously influence others.

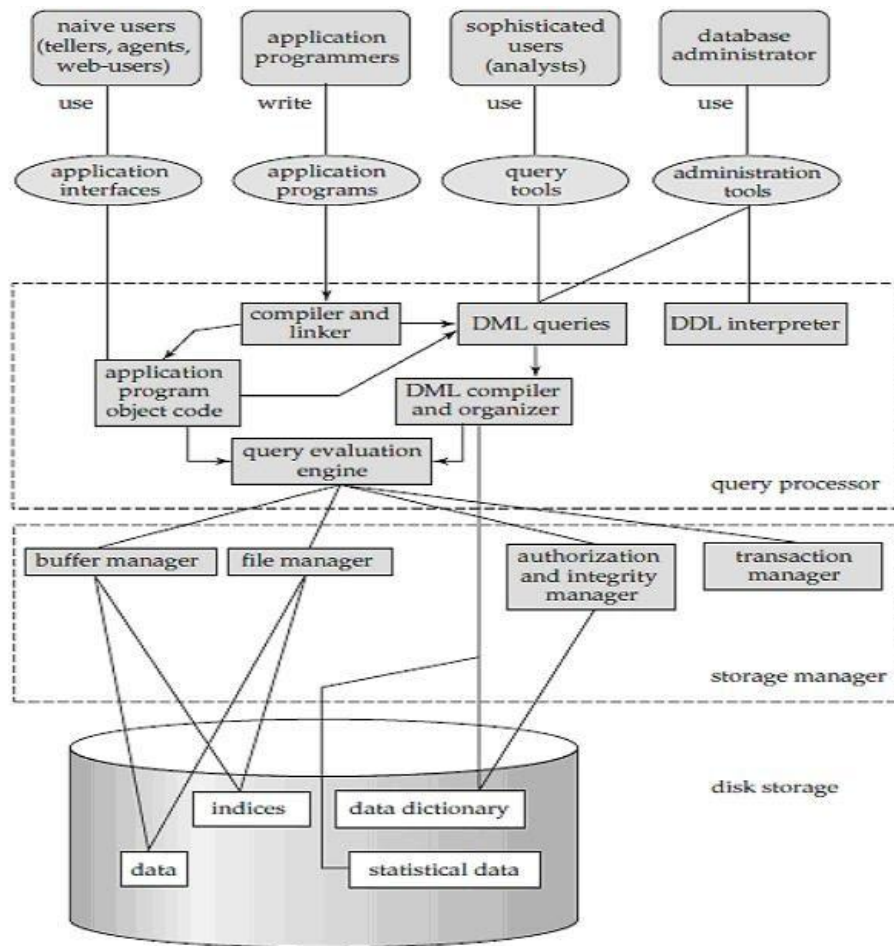
Application Architectures



- **Two-tier architecture:** E.g. client programs using ODBC/JDBC to communicate with a database
- **Three-tier architecture:** E.g. web-based applications, and applications built using “middleware”

Structure of DBMS





Database Users

- ▣ Users are differentiated by the way they expect to interact with the system
- ▣ Application programmers – interact with system through DML calls
- ▣ Sophisticated users – form requests in a database query language
- ▣ Specialized users – write specialized database applications that do not fit into the traditional data processing framework
- ▣ Naïve users – invoke one of the permanent application programs that have been written previously
 - E.g. people accessing database over the web, bank tellers, clerical staff

Database Administrator

- ▣ Coordinates all the activities of the database system; the database administrator has a good understanding of the enterprise's information resources and needs.
- ▣ Database administrator's duties include:
 - Schema definition
 - Storage structure and access method definition
 - Schema and physical organization modification
 - Granting user authority to access the database
 - Specifying integrity constraints
 - Acting as liaison with users
 - Monitoring performance and responding to changes in requirements

Data Models

- ▣ Why data models are important
- ▣ About the basic data-modeling building blocks
- ▣ What business rules are and how they affect database design
- ▣ How the major data models evolved, and their advantages and disadvantages
- ▣ How data models can be classified by level of abstraction

The Importance of Data Models

- Data model
 - Relatively simple representation, usually graphical, of complex real-world data structures
 - Communications tool to facilitate interaction among the designer, the applications programmer, and the end user
- Good database design uses an appropriate data model as its foundation

Importance of Data Modeling

- End-users have different views and needs for data
- Data model organizes data for various users

Data Model Basic Building Blocks

- Entity is anything about which data are to be collected and stored
- Attribute is a characteristic of an entity
- Relationship describes an association among (two or more) entities
 - **One-to-many (1:M) relationship**
 - **Many-to-many (M:N or M:M) relationship**
 - **One-to-one (1:1) relationship**

The Evolution of Data Models

- Hierarchical
- Network
- Relational
- Entity relationship
- Object oriented

Evolution of Major Data Models

TABLE
2.1

Evolution of Major Data Models

GENERATION	TIME	MODEL	EXAMPLES	COMMENTS
First	1960s–1970s	File system	VMS/VSAM	Used mainly on IBM mainframe systems Managed records, not relationships
Second	1970s	Hierarchical and network	IMS ADABAS IDS-II	Early database systems Navigational access
Third	Mid-1970s to present	Relational	DB2 Oracle MS SQL-Server	Conceptual simplicity Entity relationship (ER) modeling support for relational data modeling
Fourth	Mid-1980s to present	Object-oriented Extended Relational	Versant VFS/FastObjects Objectivity/DB DB/2 UDB Oracle 10g	Support complex data Extended relational products support objects and data warehousing Web databases become common
Next Generation	Present to future	XML	dbXML Tamino DB2 UDB Oracle 10g MS SQL Server	Organization and management of unstructured data Relational and object models add support for XML documents

The Hierarchical Model— Evolution

- North American Rockwell was the prime contractor for the Apollo project which culminated in the moon landing.
- This project required the management of millions of parts – a complex computer file system was used.
 - An audit of the tapes revealed that 60% of the data was redundant
 - This led Rockwell to develop an alternate strategy

The Hierarchical Model—Evolution

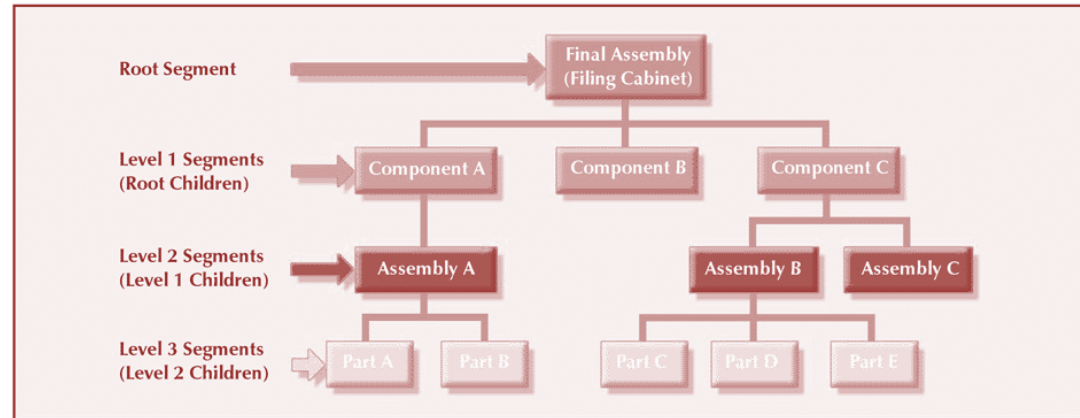
- GUAM (Generalized Update Access Method)
 - Based on the recognition that the many smaller parts would come together as components of still larger components
 - In the mid 1960s. IBM joined Rockwell to expand the capabilities of GUAM, replacing computer tapes with disk storage
- Information Management System (IMS)
 - The result of the joint effort between Rockwell and IBM
 - World's leading mainframe hierarchical database system in the 1970s and early 1980s

The Hierarchical Model— Characteristics

- Basic concepts form the basis for subsequent database development
- Limitations lead to a different way of looking at database design
- Basic concepts show up in current data models
- Best understood by examining manufacturing process

A Hierarchical Structure

FIGURE 2.1 A HIERARCHICAL STRUCTURE



Hierarchical Structure—Characteristics

- ▣ Each parent can have many children
- ▣ Each child has only one parent
- ▣ Tree is defined by path that traces parent segments to child segments, beginning from the left
- ▣ Hierarchical path
 - Ordered sequencing of segments tracing hierarchical structure
- ▣ Preorder traversal or hierarchic sequence
 - “Left-list” path
 - If Part D is most frequently accessed and updated, change the database structure to place Part D closer to the left side of the tree
 - This will give a shorter traversal

The Hierarchical Model

- Advantages over a file system
 - Conceptual simplicity – easy to understand the model layout
 - Database security
 - Data independence (a change in a data type will be automatically cascaded throughout the database by the DBMS, thereby eliminating the need to make changes in the program segments that reference the changes data type)
 - Database integrity – always a link between parent and child
 - Efficiency – very efficient when it contains a large volume of data in 1:M relationships and whose relationships are fixed over time

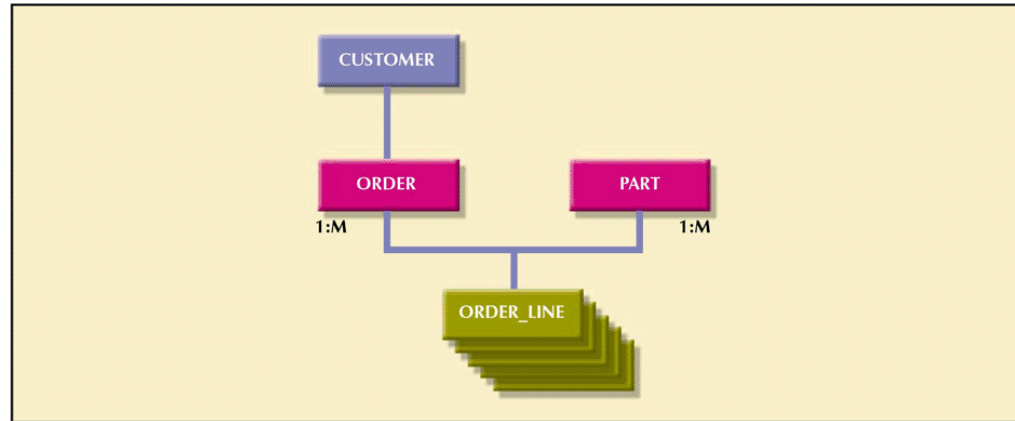
The Hierarchical Model (continued)

▣ Disadvantages over a file system

- Complex implementation – detailed knowledge of the physical data storage characteristics is required by the designers and programmers
- Difficult to manage – relocation of segments requires application changes
- Lacks structural independence
- Complex applications programming and use – programmers and end users must know precisely how the data are physically distributed within the database
- Implementation limitations – difficult to support M:N relationships
- Lack of standards – no standard DDL and no DML

Child with Multiple Parents

FIGURE 2.2 CHILD WITH MULTIPLE PARENTS



The Network Model

- Created to
 - Represent complex data relationships more effectively
 - Improve database performance
 - Impose a database standard
- Conference on Data Systems Languages (CODASYL)
- American National Standards Institute (ANSI)

Crucial Database Components

- Schema
 - Conceptual organization of entire database as viewed by the database administrator
- Subschema
 - Defines database portion “seen” by the application programs that actually produce the desired information from data contained within the database
- Data Management Language (DML)
 - Define data characteristics and data structure in order to manipulate the data

Data Management Language Components

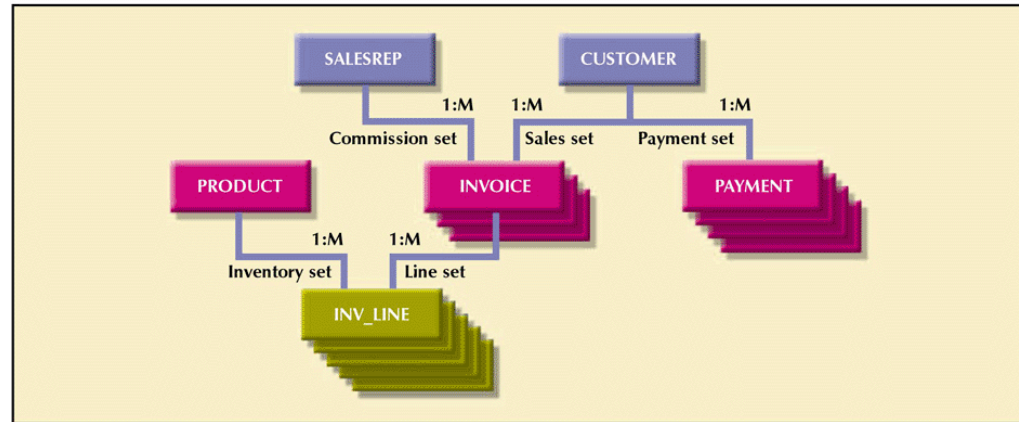
- Schema Data Definition Language (DDL)
 - Enables database administrator to define schema components
- Subschema DDL
 - Allows application programs to define database components that will be used
- DML
 - Manipulates database contents

Network Model—Basic Structure

- ▣ Resembles hierarchical model
- ▣ Collection of records in 1:M relationships
 - A relationship is called a Set
 - Composed of at least two record types
 - ▣ Owner
 - Equivalent to the hierarchical model's parent
 - ▣ Member
 - Equivalent to the hierarchical model's child
 - A record can appear as a member in more than one set i.e., a member may have multiple owners

A Network Data Model

FIGURE 2.3 A NETWORK DATA MODEL



The Network Data Model

- Advantages
 - Conceptual simplicity
 - Handles more relationship types
 - Data access flexibility – no need for a preorder traversal
 - Promotes database integrity – must first define the owner and then the member record
 - Data independence
 - Conformance to standards

The Network Data Model (continued)

- Disadvantages
 - System complexity
 - Lack of structural independence

The Relational Model

- ▣ Developed by Codd (IBM) in 1970
- ▣ Considered ingenious but impractical in 1970
- ▣ Conceptually simple
- ▣ Computers lacked power to implement the relational model
- ▣ Today, microcomputers can run sophisticated relational database software

The Relational Model: Basic Structure

- Relational Database Management System (RDBMS)
- Performs same basic functions provided by hierarchical and network DBMS systems, plus other functions
 - RDBMS handles all the complex physical details
- Most important advantage of the RDBMS is its ability to let the user/designer operate in a human logical environment

The Relational Model: Basic Structure

- Table (relations)
 - Matrix consisting of a series of row/column intersections
 - Related to each other by sharing a common entity characteristic
- Relational schema
 - Visual representation of relational database's entities, attributes within those entities, and relationships between those entities

Linking Relational Tables

FIGURE 2.4 LINKING RELATIONAL TABLES

Database name: Ch02_InsureCo Table name: AGENT (first six attributes)

	AGENT_CODE	AGENT_LNAME	AGENT_FNAME	AGENT_INITIAL	AGENT_AREACODE	AGENT_PHONE
▶	501	Alby	Alex	B	713	228-1249
	502	Hahn	Leah	F	615	882-1244
	503	Okon	John	T	615	123-5589

Link through AGENT_CODE

Table name: CUSTOMER

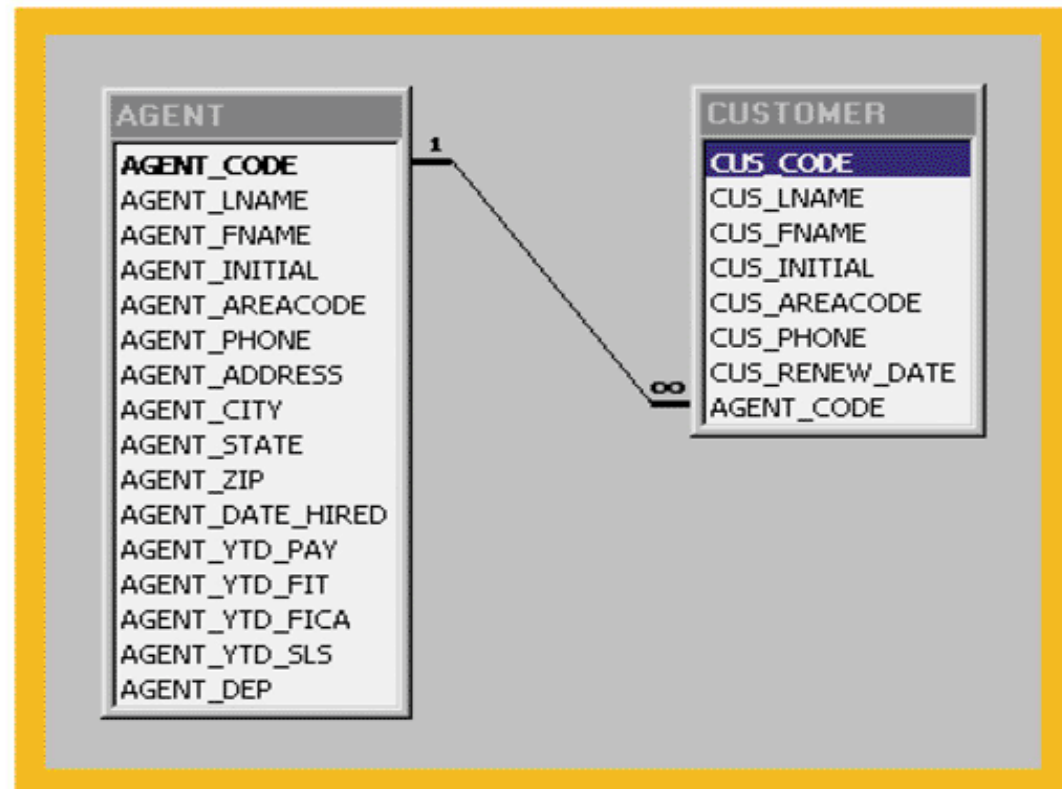
	CUS_CODE	CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_AREACODE	CUS_PHONE	CUS_RENEW_DATE	AGENT_CODE
▶	10010	Ramas	Alfred	A	615	844-2573	05-Apr-2004	502
	10011	Dunne	Leona	K	713	894-1238	16-Jun-2004	501
	10012	Smith	Kathy	W	615	894-2285	29-Jan-2005	502
	10013	Olowski	Paul	F	615	894-2180	14-Oct-2004	502
	10014	Orlando	Myron		615	222-1672	28-Dec-2004	501
	10015	O'Brian	Amy	B	713	442-3381	22-Sep-2004	503
	10016	Brown	James	G	615	297-1228	25-Mar-2004	502
	10017	Williams	George		615	290-2556	17-Jul-2004	503
	10018	Farriss	Anne	G	713	382-7185	03-Dec-2004	501
	10019	Smith	Olette	K	615	297-3809	14-Mar-2004	503

Relational Table

- Stores a collection of related entities
 - Resembles a file
- Relational table is purely logical structure
 - How data are physically stored in the database is of no concern to the user or the designer
 - This property became the source of a real database revolution

A Relational Schema

FIGURE 2.5 A RELATIONAL SCHEMA



The Relational Model

- Advantages
 - Structural independence – changes in the relational data structure do not affect the DBMS's data access in any way
 - Improved conceptual simplicity by concentrating on the logical view
 - Easier database design, implementation, management, and use
 - Ad hoc query capability - SQL
 - Powerful database management system

The Relational Model (continued)

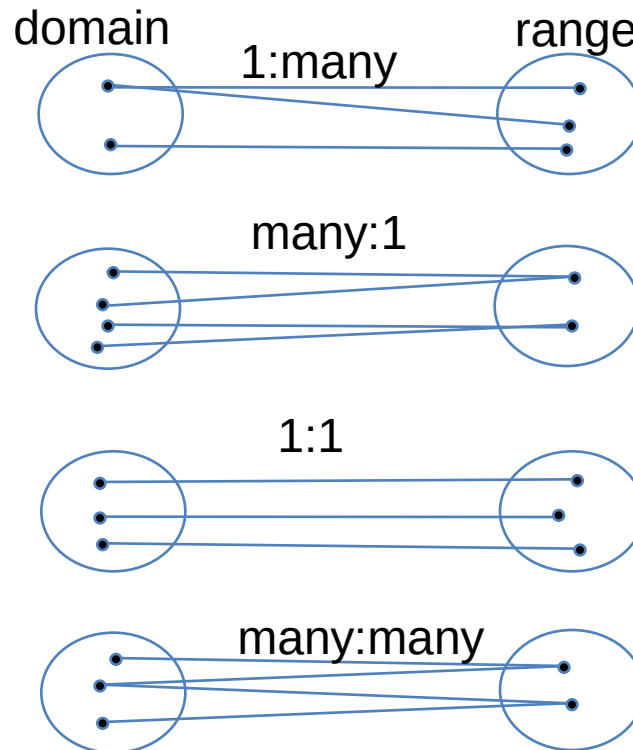
- Disadvantages
 - Substantial hardware and system software overhead
 - Can facilitate poor design and implementation
 - May promote “islands of information” problems

Relational Model

▣ Sets

- collections of items of the same type
- *no order*
- *no duplicates*

▣ Mappings



COURSE

Courseno	Subject	Lecturer	Machine
CS250	Programming	Lindsey	Sun
CS260	Graphics	Hubbold	Sun
CS270	Micros	Woods	PC
CS290	Verification	Barringer	Sun

Relational Model Notes

- ▣ no duplicate tuples in a relation
 - *a relation is a set of tuples*
- ▣ no ordering of tuples in a relation
 - *a relation is a set*
- ▣ attributes of a relation have an implied ordering
 - *but used as functions and referenced by name, not position*
- ▣ every tuple must have attribute values drawn from all of the domains of the relation or the special value *NULL*
- ▣ all a domain's values and hence attribute's values are *atomic*.

Comparative Terms

Formal	Oracle
Relation schema	Table description
Relation	Table
Tuple	Row
Attribute	Column
Domain	Value set

▣ Notation

Course (courseno, subject, equipment)

Student(studno, name, hons)

Enrol(studno, courseno, labmark)

Keys

▣ SuperKey

- a set of attributes whose values together *uniquely* identify a tuple in a relation

▣ Candidate Key

- a superkey for which no proper subset is a superkey...a key that is *minimal* .
- *Can be more than one for a relation*

▣ Primary Key

- a candidate key chosen to be the main key for the relation.
- *One for each relation*

▣ Keys can be composite

Foreign Key

- ▣ a set of attributes in a relation that exactly matches a (primary) key in another relation
 - the names of the attributes don't have to be the same but must be of the same domain
 - a foreign key in a relation A matching a primary key in a relation B represents a
- ▣ *many:one* relationship between A and B

Student(studno,name,tutor,year)

Staff(lecturer,roomno,appraiser)

Referential Integrity

- ▣ Student(studno,name,tutor,year)
- ▣ Staff(lecturer,roomno,appraiser)

- ▣ CASCADE
 - delete all matching foreign key tuples
eg. STUDENT
- ▣ RESTRICT
 - can't delete primary key tuple STAFF whilst a foreign key tuple STUDENT matches
- ▣ NULLIFY
 - foreign key STUDENT.tutor set to *null* if the foreign key ids allowed to take on null

Entity Integrity and Nulls

No part of a key can be null

▣ Attribute values

THREE categories of null values

1. Not applicable
2. Not known
3. Absent (not recorded)

■ Atomic

■ Known domain

■ **STUDENT** Sometimes can be null

studno	name	hons	tutor	year	thesis title
s1	jones	ca	bush	2	<i>null</i>
s2	brown	cis	kahn	2	<i>null</i>
s3	smith	<i>null</i>	goble	2	<i>null</i>
s4	bloggs	ca	goble	1	<i>null</i>
s5	jones	cs	zobel	1	<i>null</i>
s6	peters	ca	kahn	3	<i>null</i>

Relational Model

- ▣ General
- ▣ Simple
- ▣ Flexible
- ▣ Easy to query declaratively without programming
- ▣ But.....
 - Good design essential
 - Integrity essential
 - Poor semantics
 - Relationships based on 'value-matching'

Relational Design

stud no	name	tutor	roomno	course no	lab mark	subject
s1	jones	bush	2.26	cs250	65	programming
s1	jones	bush	2.26	cs260	80	graphics
s1	jones	bush	2.26	cs270	47	electronics
s2	brown	kahn	IT206	cs250	67	programming
s2	brown	kahn	IT206	cs270	65	electronics
s3	smith	goble	2.82	cs270	49	electronics
s4	bloggs	goble	2.82	cs280	50	design
s5	jones	zobel	2.34	cs250	0	programming
s6	peters	kahn	IT206	cs250	2	programming
<i>null</i>	<i>null</i>	capon	A14	<i>null</i>	<i>null</i>	<i>null</i>
<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>	cs290	<i>null</i>	specification
s7	patel	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>

Informal guidelines

- ▣ Semantics of the attributes
 - *easy to explain relation*
 - *doesn't mix concepts*
- ▣ Reducing the redundant values in tuples
- ▣ Choosing attribute domains that are atomic
- ▣ Reducing the null values in tuples
- ▣ Disallowing spurious tuples

Definitions

- ▣ **Cartesian Product** The cartesian product ($\times$) between n sets is the set of all possible combinations of the elements of those sets.
- ▣ **Domain** set of all possible values for an attribute; for attribute A , the domain is represented as $\text{dom}(A)$. A domain has a format and a base data type.
- ▣ **Relation Schema** denoted by $R(A_1, A_2, \dots, A_n)$, is made up of relation name R and list of attributes $A_1, A_2, \dots, A_n$.
- ▣ **Relation** a subset of the cartesian product of its domains. Given a relation schema R , a relation on that schema r , a set of attributes $A_1..A_n$ for that relation then

$$r(R) \subseteq (\text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n))$$

- ▣ **Attribute** a function on a domain for each instance of the mapping or tuple
- ▣ **Attribute Value** the result of the attribute function. Each instance of the mapping is represented by one attribute value drawn from each domain or a special NULL value. Given a tuple t and an attribute A for a relation r , $t[A] \rightarrow a$, where a is the attribute's value for that tuple

- ▣ **(N)-tuple** a set of (n) attribute-value pairs representing a single instance of a relation's mapping between its domains.
- ▣ **Degree** the number of attributes a relation has.
- ▣ **Cardinality** a number of tuples a relation has.
- ▣ **Roles** several attributes can have the same domain; the attributes indicate different *roles* in the relation.
- ▣ **Key (SuperKey)** a set of attributes whose values together *uniquely* identify every tuple in a relation. Let t_1 and t_2 be two tuples on relation r of relation schema R , and sk be a set of attributes whose values are the key for the relation schema R , then $t_1[sk] \neq t_2[sk]$.
- ▣ **(Candidate) Key** a (super)key that is *minimal*, i.e. has no proper subsets that still uniquely identify every tuple in a relation. There can be several for one relation.
- ▣ **Primary Key** a candidate key chosen to be the main key for the relation. There is only one for each relation.
- ▣ **Foreign Key** a candidate key of relation A situated in

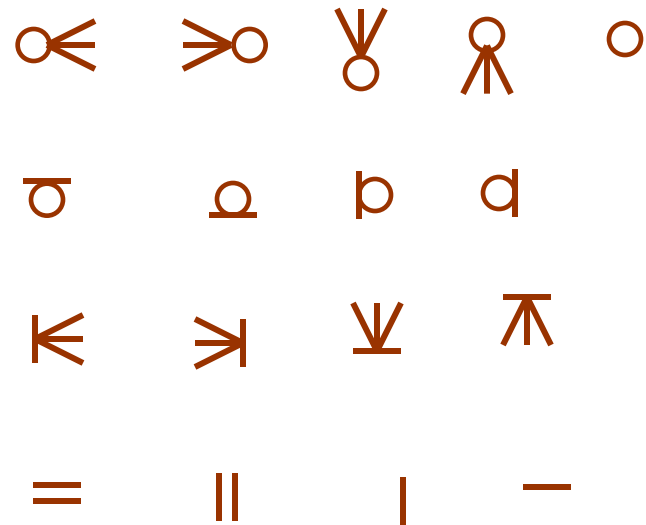
The Entity Relationship Model

- Widely accepted and adapted graphical tool for data modeling
- Introduced by Peter Chen in 1976
- Graphical representation of entities and their relationships in a database structure

The Entity Relationship Model— Basic Structure

- ▣ Entity relationship diagram (ERD)
 - Uses graphic representations to model database components
 - Entity is mapped to a relational table
- ▣ Entity instance (or occurrence) is row in table
- ▣ Entity set is collection of like entities
- ▣ Connectivity labels types of relationships
 - Diamond connected to related entities through a relationship line

The Entity Relationship Model— CrowFoot Components



Relationships: The Basic Chen ERD

A One-to-Many (1:M) Relationship: a PAINTER can paint many PAINTINGs;
each PAINTING is painted by one PAINTER



A Many-to-Many (M:N) Relationship: an EMPLOYEE can learn many SKILLs;
each SKILL can be learned by many EMPLOYEEs



A One-to-One (1:1) Relationship: an EMPLOYEE manages one STORE;
each STORE is managed by one EMPLOYEE



Relationships: The Basic Crow's Foot ERD

**A One-to-Many (1:M) Relationship: a PAINTER can paint many PAINTINGs;
each PAINTING is painted by one PAINTER**



**A Many-to-Many (M:N) Relationship: an EMPLOYEE can learn many SKILLs;
each SKILL can be learned by many EMPLOYEEs**



**A One-to-One (1:1) Relationship: an EMPLOYEE manages one STORE;
each STORE is managed by one EMPLOYEE**



The Entity Relationship Model

- Advantages
 - Exceptional conceptual simplicity
 - Visual representation
 - Effective communication tool
 - Integrated with the relational data model

The Entity Relationship Model

- Disadvantages
 - Limited constraint representation
 - Limited relationship representation
 - No data manipulation language
 - Loss of information content

The Object Oriented Model

- Modeled both data and their relationships in a single structure known as an object
- Basis of object oriented data model (OODM)
- OODM becomes the basis for the object oriented database management system (OODBMS)

The Object Oriented Model

- Object is described by its factual content
 - Like relational model's entity
- Includes information about relationships between facts within object and relationships with other objects
 - Unlike relational model's entity
- Subsequent OODM development allowed an object to also contain operations
- Object becomes basic building block for autonomous structures

Developments that Boosted OODM's Popularity

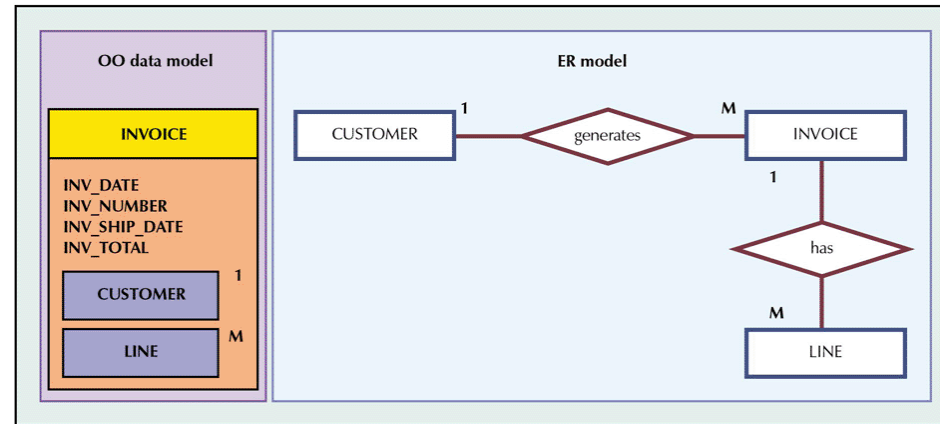
- ▣ Growing costs put a premium on code reusability
- ▣ Complex data types and system requirements became difficult to manage with a traditional RDBMS
- ▣ Became possible to support increasingly sophisticated transaction & information requirements
- ▣ Ever-increasing computing power made it possible to support the large computing overhead required

Object Oriented Data Model— Basic Structure

- ▣ Object: abstraction of a real-world entity
- ▣ Attributes describe the properties of an object
- ▣ Objects that share similar characteristics are grouped in classes
- ▣ Classes are organized in a class hierarchy
- ▣ Inheritance is the ability of an object within the class hierarchy to inherit the

A Comparison of the OO Model and the ER Model

FIGURE 2.8 A COMPARISON OF THE OO MODEL AND THE ER MODEL



The Object Oriented Model

- Advantages
 - Adds semantic content
 - Visual presentation includes semantic content
 - Database integrity
 - Both structural and data independence

The Object Oriented Model (continued)

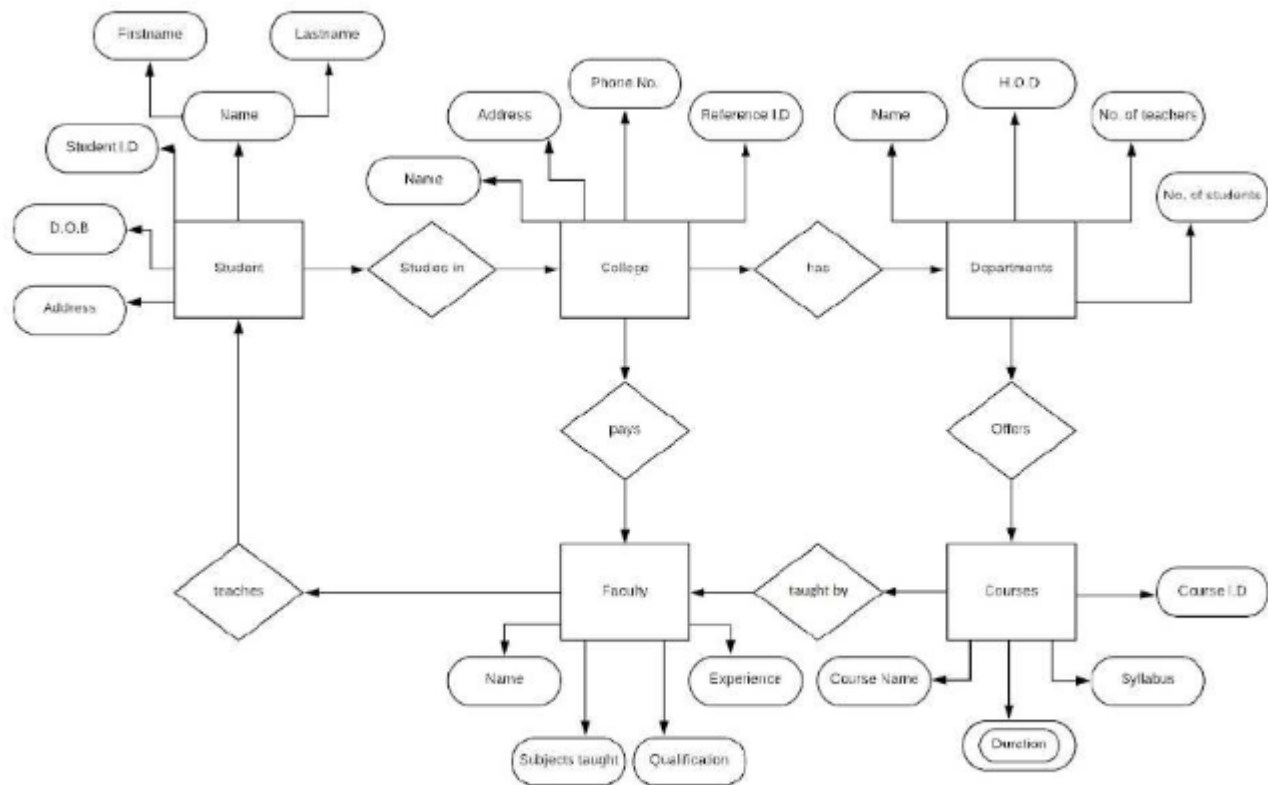
- Disadvantages
 - Slow pace of OODM standards development
 - Complex navigational data access
 - Steep learning curve
 - High system overhead slows transactions
 - Lack of market penetration

ERD CASE STUDY-1

- ▣ Construct an ER Diagram for Company having following details :
- ▣ Company organized into DEPARTMENT. Each department has unique name and a particular employee who manages the department. Start date for the manager is recorded. Department may have several locations.
- ▣ A department controls a number of PROJECT. Projects have a unique name, number and a single location.
- ▣ Company's EMPLOYEE name, ssno, address, salary, sex and birth date are recorded. An employee is assigned to one department, but may work for several projects (not necessarily controlled by her dept). Number of hours/week an employee works on each project is recorded; The immediate supervisor for the employee.
- ▣ Employee's DEPENDENT are tracked for health insurance purposes (dependent name, birthdate, relationship to employee).

PROBLEM STATEMENT:

Design a database for a college. Many students seek admission in the college. The college has a number of departments and students can be enrolled to these departments. The department also offers a number of courses to the students, each with a different duration from the other. Each department has its H.O.D and many teachers under him. The syllabus of each course is also defined. Teachers are recruited by the college for teaching the said courses to the students. The teachers may have different qualifications and experience. They may also teach different subjects if required. Each student in the college has a unique ID. We need to store the names of the students studying in the college, their residential address, date of birth. We also need to store information about the college like the name, address, contact number, reference ID, departments of the college, name of the H.O.D, number of teachers and students in the department. Also, the courses offered by the departments, the syllabus, the duration and the course ID. We can also store information about the teachers like their qualification, experience, name and subjects taught.



Other Models

- Extended Relational Data Model (ERDM)
 - Semantic data model developed in response to increasing complexity of applications
 - DBMS based on the ERDM often described as an object/relational database management system (O/RDBMS)
 - Primarily geared to business applications

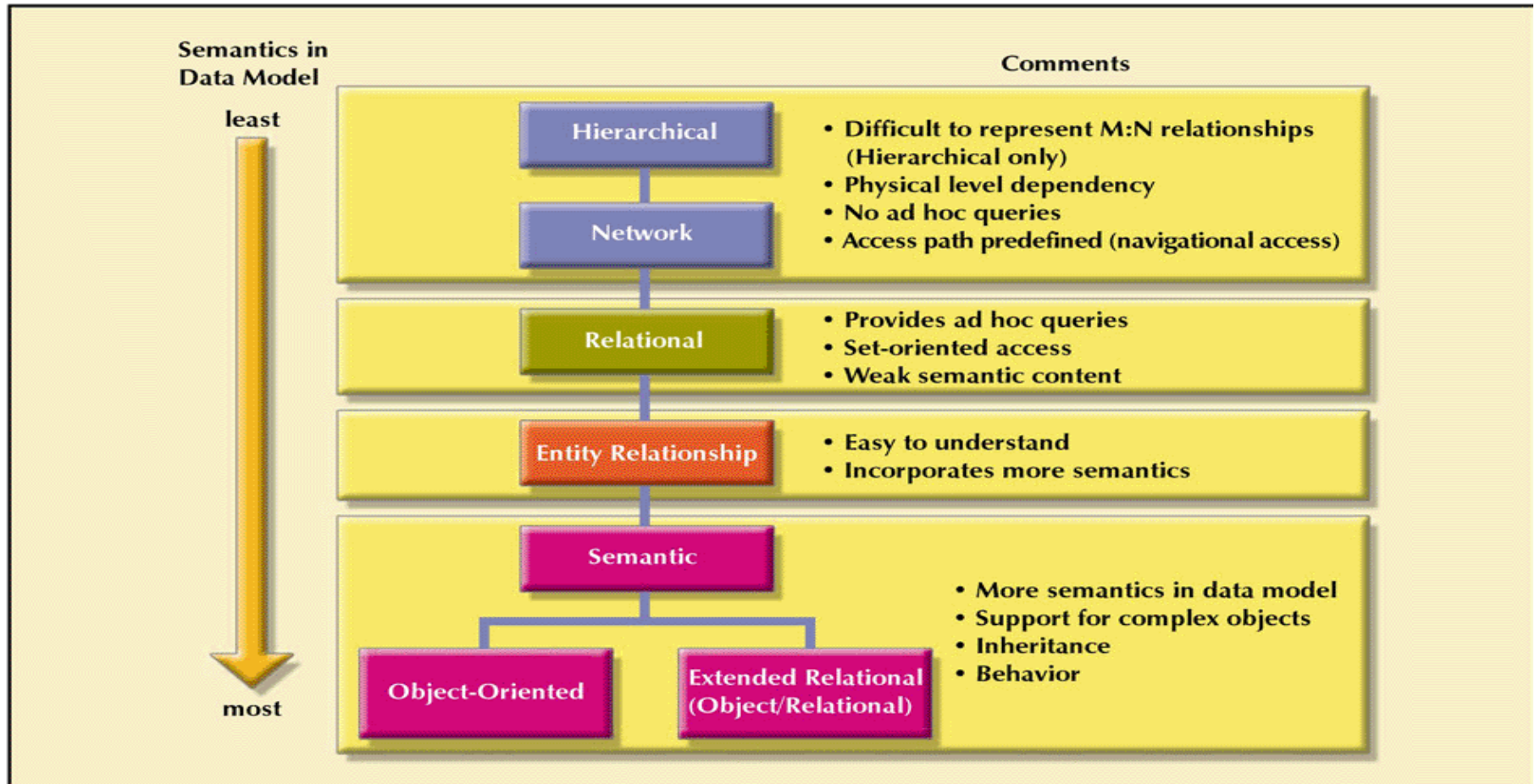
Other Models (continued)

- Date's objections to ERDM label
 - Given proper support for domains, relational data models are quite capable of handling complex data
 - Therefore, capability that is supposedly being extended is already there
 - O/RDM label is not accurate because the relational data model's domain is not an object model structure

Data Models: A Summary

- Each new data model capitalized on the shortcomings of previous models
- Common characteristics:
 - Conceptual simplicity without compromising the semantic completeness of the database
 - Represent the real world as closely as possible
 - Representation of real-world transformations (behavior) must be in compliance with consistency and integrity characteristics of any

The Development of Data Models



Rules for mapping E-R Model to Relational Model

Case 1: Binary Relationship with 1:1 cardinality with total participation of an entity

Case 2: Binary Relationship with 1:1 cardinality and partial participation of both entities

Case 3: Binary Relationship with n: 1 cardinality

Case 4: Binary Relationship with m: n cardinality

Case 5: Binary Relationship with weak entity